Database Updates

SQL Database Setup and Stored Procedures

This section creates the **Class** database, defines the main tables, and adds stored procedures for common create, read, update, and delete operations.

Database Creation

```
CREATE DATABASE Class;
USE Class;
```

Table Creation Procedures

The following procedures create the core tables used by the system. The **Class** table stores class records, while the **Student** table stores student details and links each student to a class through a foreign key.

```
CREATE PROCEDURE SP_ClassTable
AS
BEGIN
    CREATE TABLE Class(
        ClassID INT IDENTITY(1,1) PRIMARY KEY,
        ClassName VARCHAR(50) NOT NULL
    )
END;

EXEC SP_ClassTable;

CREATE PROCEDURE SP_CreateStudentTable
AS
BEGIN
    CREATE TABLE Student(
        StudentID INT IDENTITY(1,1) PRIMARY KEY,
        FirstName VARCHAR(50) NOT NULL,
        LastName VARCHAR(50) NOT NULL,
        Email VARCHAR(50),
        ClassID INT NOT NULL,
        CONSTRAINT FK_StudentClass
        FOREIGN KEY (ClassID)
        REFERENCES Class(ClassID)
    )
```

```
END;

EXEC SP_CreateStudentTable;
```

Read Procedures

These procedures retrieve class and student records either by a specific ID or by returning all records from a table.

```
CREATE PROCEDURE SP_GetClassByID
(
    @ClassID INT
)
AS
BEGIN
    SELECT * FROM Class WHERE ClassID=@ClassID;
END;

EXEC SP_GetClassByID
    @ClassID=1;

CREATE PROCEDURE SP_GetAllClasses
AS
BEGIN
    SELECT * FROM Class;
END;

EXEC SP_GetAllClasses;

CREATE PROCEDURE SP_GetStudentByID
(
    @StudentID INT
)
AS
BEGIN
    SELECT * FROM Student WHERE StudentID=@StudentID;
END;

EXEC SP_GetStudentByID
    @StudentID=1;
```

```
CREATE PROCEDURE SP_GetAllStudents
AS
BEGIN
    SELECT * FROM Student;
END;

EXEC SP_GetAllStudents;
```

Insert Procedures

These procedures add new class and student records to the database.

```
CREATE PROCEDURE SP_AddClass
(
    @ClassName VARCHAR(50)
)
AS
BEGIN
    INSERT INTO Class(
        ClassName
    )
    VALUES
    (
        @ClassName
    );
END;

EXEC SP_AddClass
    @ClassName="HR";

CREATE PROCEDURE SP_AddStudent
(
    @FirstName VARCHAR(50),
    @LastName VARCHAR(50),
    @Email VARCHAR(50),
    @ClassID INT
)
AS
BEGIN
    INSERT INTO Student(
        FirstName,
```

```
        LastName,
        Email,
        ClassID
    ) VALUES (
        @FirstName,
        @LastName,
        @Email,
        @ClassID
    );
END;

EXEC SP_AddStudent
    @FirstName="Sriven",
    @LastName="B",
    @Email="S@gmail.com",
    @ClassID=1;
```

Delete Procedures

These procedures remove class or student records by ID.

```
CREATE PROCEDURE SP_DeleteClassByID
(
    @ClassID INT
)
AS
BEGIN
    DELETE FROM Class WHERE ClassID=@ClassID;
END;

EXEC SP_DeleteClassByID
    @ClassID=1;

CREATE PROCEDURE SP_DeleteStudentByID
(
    @StudentID INT
)
AS
BEGIN
    DELETE FROM Student WHERE StudentID=@StudentID;
END;
```

```
EXEC SP_DeleteStudentByID
    @StudentID=1;
```

Update Procedures

These procedures update existing class and student records by ID.

```
CREATE PROCEDURE SP_UpdateClassByID
(
    @ClassName VARCHAR(50),
    @ClassID INT
)
AS
BEGIN
    UPDATE Class
    SET ClassName=@ClassName
    WHERE ClassID=@ClassID;
END;

EXEC SP_UpdateClassByID
    @ClassName="BIPC",
    @ClassID=1;

CREATE PROCEDURE SP_UpdateStudentByID
(
    @FirstName VARCHAR(50),
    @LastName VARCHAR(50),
    @Email VARCHAR(50),
    @StudentID INT
)
AS
BEGIN
    UPDATE Student
    SET FirstName=@FirstName,
        LastName=@LastName,
        Email=@Email
    WHERE StudentID=@StudentID;
END;

EXEC SP_UpdateStudentByID
```

```
    @FirstName="Ramu",
    @LastName="Bantu",
    @Email="Sb@gmail.com",
    @StudentID=2;
```

Employee Controller with DTO

The **EmployeesController** handles employee API requests. It uses **ApplicationDbContext** to work with the database and uses **AddEmployeeDto** to receive only the data needed when adding a new employee.

Key Parts of the Controller

- **Route:** Defines the API path as **api/employees**.
- **DbContext:** Provides access to the Employee table in the database.
- **GET method:** Retrieves all employee records.
- **POST method:** Accepts employee data through **AddEmployeeDto**, converts it into an **Employee** entity, and saves it to the database.

Controller Code Example

```
using EmployeeManagementAPI.Data;
using EmployeeManagementAPI.Model;
using EmployeeManagementAPI.Model.Entities;
using Microsoft.AspNetCore.Mvc;

namespace EmployeeManagementAPI.Controllers
{
    // localhost:xxxx/api/employees
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeesController : ControllerBase
    {
        private readonly ApplicationDbContext dbContext;

        public EmployeesController(ApplicationDbContext dbContext)
        {
            this.dbContext = dbContext;
        }

        [HttpGet]
        public IActionResult GetAllEmployees()
```

```
        {
            var allEmployees = dbContext.Employee.ToList();
            return Ok(allEmployees);
        }

        [HttpPost]
        public IActionResult AddEmployee(AddEmployeeDto
addEmployeeDto)
        {
            var employeeEntity = new Employee()
            {
                FirstName = addEmployeeDto.FirstName,
                LastName = addEmployeeDto.LastName,
                Email = addEmployeeDto.Email
            };

            dbContext.Employee.Add(employeeEntity);
            dbContext.SaveChanges();

            return Ok(employeeEntity);
        }
    }
}
```

DTOs for Separation

A **DTO**, or **Data Transfer Object**, is used to separate the data exposed by the application from the actual database entity classes. Instead of sending or receiving the full entity model directly, the application can use DTO classes to control which fields are included in requests and responses.

Entity

An entity represents the full database table structure.

DTO

A DTO represents the data used in an API request or response. It contains only the fields the API needs.

Why Use a DTO?

DTOs help avoid exposing extra database fields. They also let the API send and receive only the data required for a specific operation.

Example

When adding a student, the user should enter the name, email, and class ID. The user should not enter the student ID because the database creates it automatically.

AddEmployeeDto Class

The **AddEmployeeDto** class is used when adding a new employee through the API. It includes only the fields that the API needs from the user.

- **FirstName:** Required value for the employee's first name.
- **LastName:** Required value for the employee's last name.
- **Email:** Optional value because it uses nullable string syntax.

DTO Code Example

```
namespace EmployeeManagementAPI.Model
{
    public class AddEmployeeDto
    {
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
    }
}
```

ClassManagementAPI

Student Entity Overview

The **Student** entity defines the structure of the student table in the database. Each property in the class represents a column in that table.

- **StudentID:** Stores the unique identifier for each student.
- **FirstName:** Stores the student's first name and is required.
- **LastName:** Stores the student's last name and is required.
- **Email:** Stores the student's email address and is optional.

Student Entity Code Example

```
namespace ClassManagementAPI.Models.Entities
{
```

```
    public class Student
    {
        public Guid StudentID { get; set; }
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
    }
}
```

ClassManagementAPI DbContext, DTO, and Controller

This section shows how the ClassManagementAPI project connects the Student entity to Entity Framework Core, uses a DTO for API input, and exposes controller methods for retrieving and adding students.

ApplicationDbContext

The **ApplicationDbContext** class connects the application to the database. The **Students** DbSet represents the Student table.

```
using ClassManagementAPI.Models.Entities;
using Microsoft.EntityFrameworkCore;

namespace ClassManagementAPI.Data
{
    public class ApplicationDbContext : DbContext
    {
        public
ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Student> Students { get; set; }
    }
}
```

Student DTO

The **StudentDto** class is used to receive student data from the API. It keeps the API input separate from the database entity.

```
namespace ClassManagementAPI.Models
{
    public class StudentDto
    {
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
    }
}
```

Students Controller

The **StudentsController** handles API requests for student records. It uses **ApplicationDbContext** to connect to the database and **StudentDto** to receive only the required input when adding a new student.

- **GET method:** Retrieves all student records from the database.
- **POST method:** Accepts student data through **StudentDto**, converts it into a **Student** entity, saves it to the database, and returns the created student.

EmployeeManagementAPI Overview

This section outlines the main parts of the **EmployeeManagementAPI** project, including the employee entity, application settings, startup configuration, DTO classes, and controller actions for employee records.

Employee Entity

The **Employee** entity represents the employee table structure. Each property maps to a database column and defines the information stored for each employee.

- **EmployeeID:** Unique identifier for each employee.
- **FirstName:** Required employee first name.
- **LastName:** Required employee last name.
- **Email:** Optional employee email address.

Employee Entity Code Example

```
namespace EmployeeManagementAPI.Models.Entities
{
    public class Employee
    {
        public Guid EmployeeID { get; set; }
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
    }
}
```

Application Settings

The application settings file defines logging behavior, allowed hosts, and the SQL Server connection string used by Entity Framework Core.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection":
"Server=.;Database=EmployeeDB;Trusted_Connection=true;TrustServerCerti
ficate=true;"
  }
}
```

Program Configuration

The **Program.cs** file configures the main services and request pipeline for the EmployeeManagementAPI application. It registers controllers, enables Swagger for API testing, connects Entity Framework Core to SQL Server, and maps controller endpoints.

```
using EmployeeManagementAPI.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Register application services.
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Configure Entity Framework Core to use the SQL Server connection
string.
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("De
faultConnection")));

var app = builder.Build();

// Enable Swagger in the development environment.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

// Configure the HTTP request pipeline.
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Employee DTO Classes

DTO classes define the data shape used by the API when sending or receiving employee information. They help keep API input and output separate from the database entity.

EmployeeDto

The **EmployeeDto** class represents the full employee data returned by the API, including the employee ID.

```
namespace EmployeeManagementAPI.Models
{
    public class EmployeeDto
    {
        public Guid EmployeeID { get; set; }
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
    }
}
```

UpdateEmployeeDto

The **UpdateEmployeeDto** class contains only the editable employee fields used when updating an existing record.

```
namespace EmployeeManagementAPI.Models
{
    public class UpdateEmployeeDto
    {
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
    }
}
```

Employee Controller CRUD Operations

The **EmployeeController** manages the main API operations for employee records. It uses **ApplicationDbContext** to access the database, **EmployeeDto** when adding employees, and **UpdateEmployeeDto** when updating existing records.

Controller Responsibilities

- **GET all employees:** Retrieves every employee record from the database.
- **POST employee:** Creates a new employee from the data provided in **EmployeeDto**.
- **GET employee by ID:** Finds and returns one employee by GUID.
- **PUT employee by ID:** Updates an existing employee using **UpdateEmployeeDto**.
- **DELETE employee by ID:** Removes an employee record from the database.

Summary of Common Entity Framework Methods

The controller uses several common Entity Framework Core methods to retrieve, add, update, and delete employee records. These methods help the API interact with the database through the **ApplicationDbContext**.

- **ToList():** Converts the employee query into a list and returns all employee records from the database.
- **Find(id):** Searches the employee table for a record with the matching ID.
- **Add(employeeEntity):** Adds a new employee object to the database context before it is saved.
- **Remove(employee):** Marks an existing employee record for deletion from the database.
- **SaveChanges():** Saves all pending changes, such as adding, updating, or deleting records, to the database.

Controller Code Example

```
using EmployeeManagementAPI.Data;
using EmployeeManagementAPI.Models;
using EmployeeManagementAPI.Models.Entities;
using Microsoft.AspNetCore.Mvc;

namespace EmployeeManagementAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        private readonly ApplicationDbContext _dbcontext;

        public EmployeeController(ApplicationDbContext dbcontext)
        {
            _dbcontext = dbcontext;
        }

        [HttpGet]
        public IActionResult GetAllEmployee()
        {
            var employees = _dbcontext.Employees.ToList();
            return Ok(employees);
```

```
        }

        [HttpPost]
        public IActionResult AddEmployee(EmployeeDto employeedto)
        {
            var employeeEntity = new Employee
            {
                FirstName = employeedto.FirstName,
                LastName = employeedto.LastName,
                Email = employeedto.Email
            };

            _dbcontext.Employees.Add(employeeEntity);
            _dbcontext.SaveChanges();
            return Ok(employeeEntity);
        }

        [HttpGet]
        [Route("{id:Guid}")]
        public IActionResult GetEmployeeByID(Guid id)
        {
            var employee = _dbcontext.Employees.Find(id);

            if (employee is null)
            {
                return NotFound();
            }

            return Ok(employee);
        }

        [HttpPut]
        [Route("{id:Guid}")]
        public IActionResult UpdateEmployeeByID(Guid id,
UpdateEmployeeDto employeeDto)
        {
            var employee = _dbcontext.Employees.Find(id);

            if (employee is null)
```

```
            {
                return NotFound();
            }

            employee.FirstName = employeeDto.FirstName;
            employee.LastName = employeeDto.LastName;
            employee.Email = employeeDto.Email;

            _dbcontext.SaveChanges();
            return Ok(employee);
        }

        [HttpDelete]
        [Route("{id:Guid}")]
        public IActionResult DeleteEmployeeByID(Guid id)
        {
            var employee = _dbcontext.Employees.Find(id);

            if (employee is null)
            {
                return NotFound();
            }

            _dbcontext.Employees.Remove(employee);
            _dbcontext.SaveChanges();
            return Ok(employee);
        }
    }
}
```

ClassManagement Student API Structure

This section explains the main parts of the **ClassManagement** student API, including the Student entity, DTO classes, and controller methods used to perform CRUD operations.

Student Entity

The **Student** entity represents the student table in the database. Each property maps to a column and stores basic student information.

- **StudentId:** Unique identifier for each student.
- **FirstName:** Required first name of the student.
- **LastName:** Required last name of the student.
- **Email:** Optional email address for the student.

```
namespace ClassManagement.Models.Entities
{
    public class Student
    {
        public Guid StudentId { get; set; }
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
    }
}
```

Student DTO Classes

DTO classes define the data accepted by the API. They keep request data separate from the full database entity and make the controller easier to maintain.

StudentDto

The **StudentDto** class is used when adding a new student. It includes only the fields supplied by the user.

```
namespace ClassManagement.Models
{
    public class StudentDto
    {
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
    }
}
```

UpdateStudentDto

The **UpdateStudentDto** class is used when updating an existing student record. It contains only the editable student fields.

```
namespace ClassManagement.Models
{
    public class UpdateStudentDto
    {
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
    }
}
```

Student Controller CRUD Operations

The **StudentController** handles API requests for student records. It uses **ApplicationDbContext** to access the database and DTO classes to receive input for adding or updating students.

Controller Responsibilities

- **GET all students:** Retrieves every student record from the database.
- **GET student by ID:** Finds and returns one student by GUID.
- **POST student:** Creates a new student from the data provided in **StudentDto**.
- **PUT student by ID:** Updates an existing student using **UpdateStudentDto**.
- **DELETE student by ID:** Removes a student record from the database.

Controller Code Example

```
using ClassManagement.Data;
using ClassManagement.Models;
using ClassManagement.Models.Entities;
using Microsoft.AspNetCore.Mvc;

namespace ClassManagement.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentController : ControllerBase
    {
        private readonly ApplicationDbContext _dbContext;

        public StudentController(ApplicationDbContext dbContext)
```

```
        {
            _dbContext = dbContext;
        }

        [HttpGet]
        public IActionResult GetAllStudents()
        {
            var students = _dbContext.Students.ToList();
            return Ok(students);
        }

        [HttpGet]
        [Route("{id:Guid}")]
        public IActionResult GetStudentById(Guid id)
        {
            var student = _dbContext.Students.Find(id);

            if (student is null)
            {
                return NotFound();
            }

            return Ok(student);
        }

        [HttpPost]
        public IActionResult AddStudent(StudentDto studentDto)
        {
            var student = new Student
            {
                FirstName = studentDto.FirstName,
                LastName = studentDto.LastName,
                Email = studentDto.Email
            };

            _dbContext.Students.Add(student);
            _dbContext.SaveChanges();
            return Ok(student);
        }
```

```
        [HttpPut]
        [Route("{id:Guid}")]
        public IActionResult UpdateStudentById(Guid id,
UpdateStudentDto updateStudentDto)
        {
            var student = _dbContext.Students.Find(id);

            if (student is null)
            {
                return NotFound();
            }

            student.FirstName = updateStudentDto.FirstName;
            student.LastName = updateStudentDto.LastName;
            student.Email = updateStudentDto.Email;

            _dbContext.SaveChanges();
            return Ok(student);
        }

        [HttpDelete]
        [Route("{id:Guid}")]
        public IActionResult DeleteStudentById(Guid id)
        {
            var student = _dbContext.Students.Find(id);

            if (student is null)
            {
                return NotFound();
            }

            _dbContext.Students.Remove(student);
            _dbContext.SaveChanges();
            return Ok(student);
        }
    }
}
```